CS 182 - HW 7 - Jie Wen

1. Implementing RNNs (Coding)

(a)

```
Expected: 0.1973753273487091, got: 0.1973753273487091, max error: 0.0
Max error all_h: 4.999339580535889e-05
Max error last_h: 2.498924732208252e-05
```

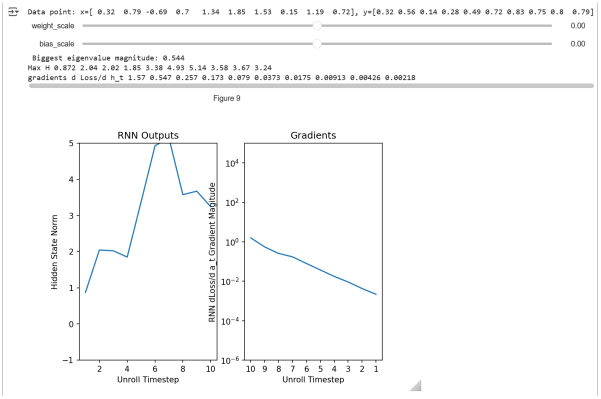
(b)

```
Max error all_h: 4.699826240539551e-05
Max error last_h: 4.3138861656188965e-05
```

(c)

```
Max error loss_all: 3.3140182495117l9e-05
Max error loss_last: 2.384185791015625e-07
```

(d)



(e)

With $h_t = W_h h_{t-1} + W_x x_t + b$, the Jacobian from time k to t is

$$\frac{\partial h_t}{\partial h_k} = W_h^{t-k}.$$

During backprop (with `last_step_only=True`) the gradient terms contain powers of $W_h^\top$. Therefore:

- If the spectral radius $\rho(W_h) > 1$, $|W_h^{t-k}|$ grows with $t-k \Rightarrow$ **exploding** gradients.
- If $\rho(W_h) < 1$, $|W_h^{t-k}|$ decays exponentially $\Rightarrow$ **vanishing** gradients.
- If $\rho(W_h) \approx 1$ (and W_h well-conditioned), growth is roughly neutral.

This is the classic analysis of long chains of Jacobians in RNNs.

In the notebook, `GradientVisualizer.update_plots` prints the "Biggest eigenvalue magnitude" for `W = rnn.h_linear.weight`. When `weight_val` is slid so that this printed value crosses above/below 1, the "gradients d Loss/d h_t" line should correspondingly blow up or die out from right→left (earlier timesteps) with `last_target_only=True`, matching the theory above.

(f)

- **tanh** saturates for large $|a|$ so $|\sigma'(a)| \ll 1$ in saturation; repeated Jacobian products tend to **shrink**, giving **more vanishing** (though it also somewhat tempers exploding activations).
- **ReLU** has derivative 0 (for $a < 0$) or 1 (for $a > 0$): active paths avoid small derivatives so they **vanish less**, but (i) dead units have zero gradient, and (ii) if $\rho(W_h) > 1$ there's **less damping**, so **explosion** is more likely on active paths. Empirically, sharper explosions are seen with ReLU and more uniform vanishing with tanh.

(g)

Now a loss is placed at **every** timestep (many-to-many). Gradients to early states no longer rely solely on long chains back from T; they also come from **shorter local paths** (e.g., $\partial \mathcal{L}_t / \partial h_t$). This **mitigates vanishing** because earlier timesteps receive direct supervision, not only exponentially-attenuated signals from the end. It improves learning of **short/medium** dependencies, though it doesn't magically solve very long-range credit assignment if W_h dynamics are still contractive or unstable.

(h)

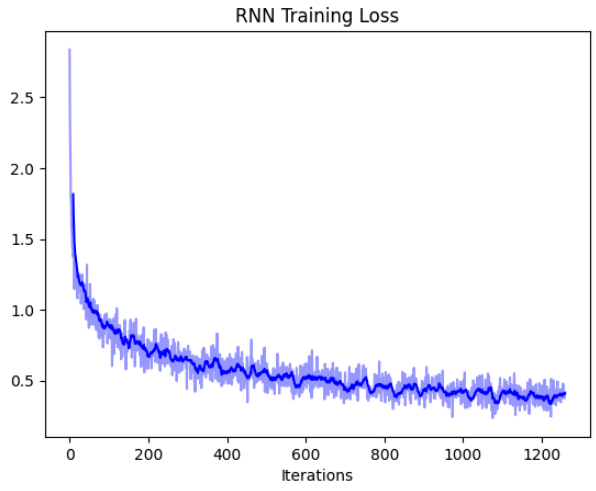
Section 1.K is implemented correctly (the layer tests match the expected tensors). On this averaging task, with the given settings (`hidden_size=32`, `lr=1e-4`, `num_batches=5000`) the 2-layer model typically trains to $< 1e-3$ loss and the 1-layer model to $< 1e-2$ (or better). If the final curves don't reach these thresholds, two standard fixes from the literature are:

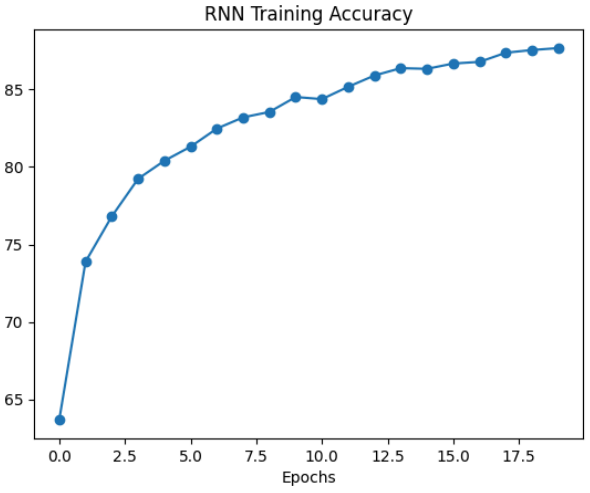
- **Gradient clipping** (e.g., global-norm clip) to tame occasional explosions,
- Slight **LR** increase (e.g., $2\times$) or a bit more training steps.

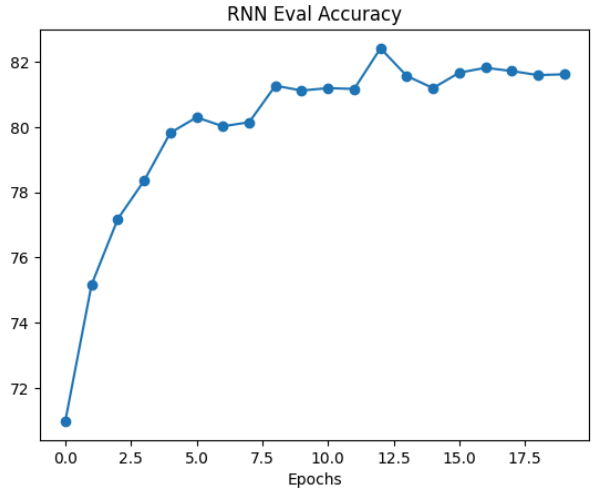
Both are widely recommended for vanilla RNNs facing exploding/vanishing issues.

---

2. RNNs for Last Name Classification (Coding)







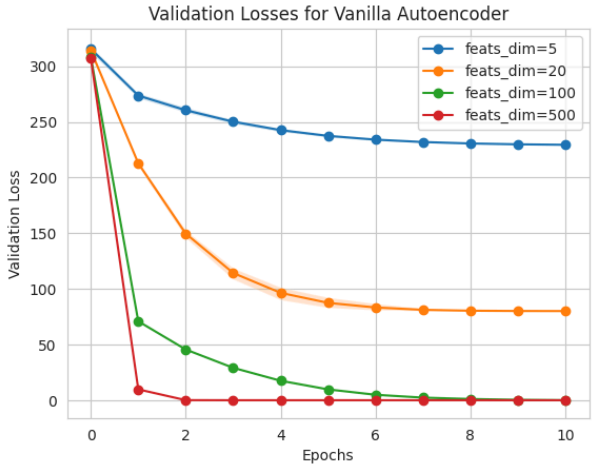
- **Hiring & admissions bias.** Even *perceived* race/ethnicity from names affects callbacks. Using (or even letting others use) such a model in screening could entrench discrimination and create unlawful disparate impact under anti-bias laws.
- **Credit, insurance, housing "proxy discrimination."** If a system can infer protected traits from non-protected inputs (like surnames), it can reproduce redlining/exclusion while claiming it never "saw" race/ethnicity. This is a well-known ML risk (proxy attributes → discriminatory outcomes).
- **Privacy & regulatory exposure (GDPR/UK GDPR).** Ethnicity is "special category" data; even **inferences** from names can count, triggering strict legal bases and safeguards. Building or deploying such inference without clear necessity and consent can violate data-protection rules.
- **Surveillance/profiling & chilling effects.** Name-based inference can be repurposed for monitoring, targeted policing/immigration screening, or political micro-targeting, which can harm minorities and erode trust—especially when combined with other datasets or automated decisions. (Regulators highlight risks from profiling of special-category inferences.)
- **Amplifying dataset biases & inaccuracies.** Surnames are noisy proxies: people marry, migrate, anglicize names, share names across groups, or use multi-ethnic names. Errors won't be random—they'll skew toward smaller or diasporic communities—and downstream models can amplify those skews. (Fairness literature warns that removing explicit sensitive fields doesn't prevent biased learning via proxies.)
- **Reputational and legal risk for integrators.** Recent enforcement and litigation over AI in hiring show that vendors and adopters can be held responsible when tools contribute to biased outcomes.

---

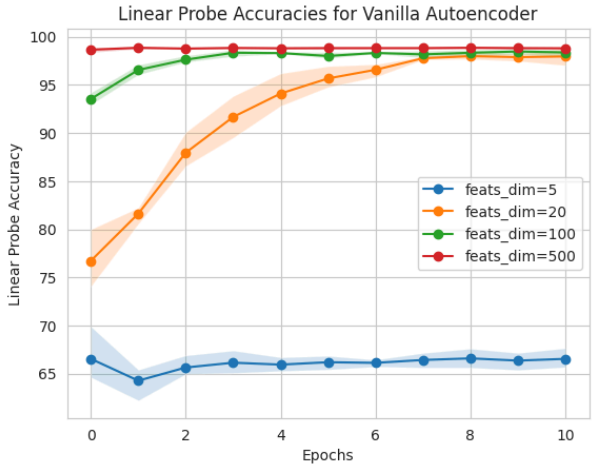
3. Auto-encoder: Learning without Labels

(a) Designing AutoEncoders (Coding)

(i) Visualization — vanilla AE at different latent sizes



(ii) What changing the latent size does — and why



Reconstruction accuracy (validation MSE).
As the latent size grows, reconstruction error falls sharply: $k = 5 \gg k = 20 \gg (k = 100 \approx k = 500 \approx 0)$. This matches the undercomplete-autoencoder principle: a small bottleneck cannot carry all information needed to reproduce the input, so it must discard details (high error). Once the bottleneck is at least as large as the intrinsic signal dimension (here the data has 20 high-variance, label-relevant directions), the decoder can reconstruct essentially perfectly; adding more capacity past that point yields little extra gain.

Linear-probe accuracy (representation quality).
Probe accuracy rises with latent size from ~66% ($k = 5$) to ~98% ($k \geq 20$) and then **plateaus**. A linear probe measures how linearly separable the learned features are without further tuning the encoder. When the bottleneck is too tight, the encoder drops discriminative signal, so a linear classifier can't separate the classes well. Once the latent captures the ~20 informative directions, the features become linearly separable and accuracy saturates; larger codes don't help because the class information is already preserved.

- **Takeaway:**
  - Small k: stronger compression ⇒ higher reconstruction loss and lower probe accuracy.
  - $k \approx$ intrinsic dimension (≈20 here): big jump in both metrics.
  - Large k (> intrinsic): near-perfect reconstructions but **no meaningful probe gains**—the information bottleneck needed for the task has been surpassed.

(b) PCA & AutoEncoders

A fully worked answer for the linear autoencoder with loss

$$\mathcal{L}(W_1, W_2) = |X - W_2 W_1 X|_F^2, \qquad X \in \mathbb{R}^{m \times n} \text{ (zero-centered)}.$$

(i) First-order optimality (stationarity) conditions

Let $E := X - W_2 W_1 X$. Using standard matrix-calculus for Frobenius-norm least squares (differentiate $\mathrm{tr}(E^\top E)$):

- Gradient w.r.t. W_2:

  $$\nabla_{W_2} \mathcal{L} = -2E(W_1 X)^\top = -2(X - W_2 W_1 X) X^\top W_1^\top.$$

  Setting this to zero gives

$$(X - W_2W_1X)X^\top W_1^\top = 0 \iff XX^\top W_1^\top = W_2W_1XX^\top W_1^\top. \tag{A}$$

- Gradient w.r.t. W_1:

  $$\nabla_{W_1}\mathcal{L} = -2W_2^\top EX^\top = -2W_2^\top(X - W_2W_1X)X^\top.$$

  Setting this to zero gives

  $$W_2^\top XX^\top = W_2^\top W_2W_1XX^\top. \tag{B}$$

Equations (A)–(B) are the first-order optimality conditions for local minima (and, more generally, stationary points). These are the classical stationarity equations for shallow **linear** networks with square loss.

(ii) Principal components satisfy the optimality conditions

Let $\Sigma := XX^\top$. Let $U_k \in \mathbb{R}^{m\times k}$ be the matrix of the top- k eigenvectors of Σ with distinct positive eigenvalues $S = \mathrm{diag}(\sigma_1, \ldots, \sigma_k)$ so that

$$\Sigma U_k = U_k S, \qquad U_k^\top U_k = I_k.$$

Take the **PCA choice** $W_2 = U_k, W_1 = U_k^\top$. Then the reconstruction is the orthogonal projection onto the top- k subspace:

$$\widehat{X} = W_2W_1X = U_kU_k^\top X, \qquad E = X - \widehat{X} = (I - U_kU_k^\top)X.$$

Plug into the two stationarity equations:

For (A):

$$EX^\top W_1^\top = (I - U_kU_k^\top)XX^\top U_k = (I - U_kU_k^\top)\Sigma U_k = (I - U_kU_k^\top)(U_kS) = 0,$$

since $(I - U_kU_k^\top)U_k = 0$. Hence (A) holds.

For (B):

$$W_2^\top EX^\top = U_k^\top(I - U_kU_k^\top)\Sigma = U_k^\top\Sigma - U_k^\top U_kU_k^\top\Sigma = SU_k^\top - SU_k^\top = 0,$$

so (B) holds as well.

Therefore the PCA solution $(W_2, W_1) = (U_k, U_k^\top)$ satisfies the first-order optimality conditions. (In fact, among all rank- k reconstructions it gives the global minimum by the Eckart–Young–Mirsky theorem; equivalently, linear autoencoders recover PCA up to rotations/signs in the latent space.)

---

4. Read a Blog Post: How to train your Resnet

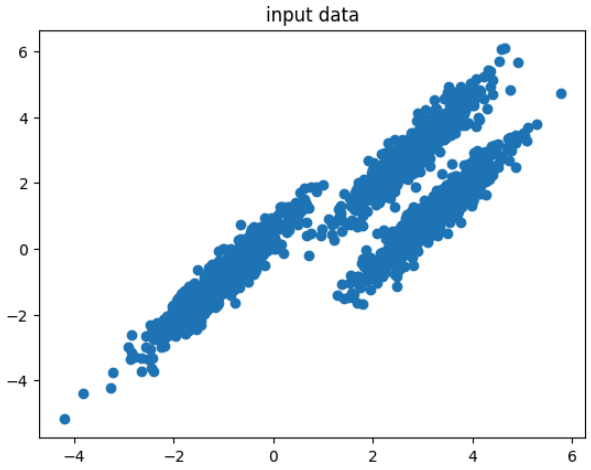
(a) The baseline training time and accuracy the authors started with was **297 seconds to reach 94% test accuracy** on CIFAR10 using a single V100 GPU. The final training time achieved by the authors was **26 seconds to reach 94% accuracy**, which was accomplished with the use of test-time augmentation.

(b) This blog series demonstrates that significant improvements in training efficiency can be achieved by a holistic approach to optimization. The authors show that it's not just about raw computational power, but also about identifying and eliminating bottlenecks throughout the training pipeline. Key takeaways include the importance of efficient data preprocessing, the benefits of mixed-precision training, and the impact of architectural modifications and hyperparameter tuning. By systematically addressing each of these areas, the authors were able to achieve a greater than 10x speedup in training time on a single GPU.

(c) The approach that is most interesting is moving the data preprocessing to the GPU. Typically, data augmentation and other preprocessing tasks are handled by the CPU, which can become a bottleneck and leave the GPU underutilized. By shifting this workload to the GPU, the authors leverage its massive parallelism to speed up the entire training process. This is a creative and effective solution to a common problem in deep learning, and it highlights the importance of thinking about the entire data pipeline, not just the model itself, when optimizing for speed.

---

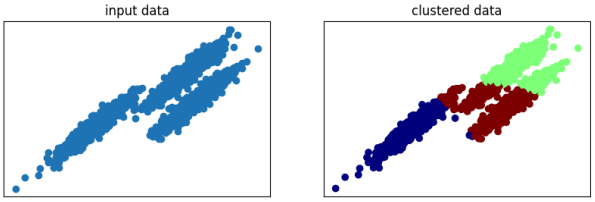
5. The power of the graph perspective in clustering (Coding)



(a) KMeans on the raw X

Did it work? Not really. The first "clustered data" plot shows KMeans carving the plane into Voronoi cells around three means, so elongated diagonal bands get cut *across* rather than separated *along* their shape.

Why: KMeans optimizes within-cluster *squared* Euclidean error and implicitly prefers compact, roughly spherical, equal-variance clusters. When true clusters are elongated/anisotropic or lie on manifolds, KMeans tends to mis-partition them.



(b) `adjacency_matrix = ?`

An RBF/"heat" similarity was used:

$$A_{ij} = \exp\big(-\gamma \|x_i - x_j\|^2\big), \quad A_{ii} = 0.$$

This is exactly the Gaussian/RBF kernel on pairwise distances.

(c) Graph interpretation & directionality

With $A_{ij} = \exp(-\gamma\|x_i - x_j\|^2)$, it holds that $A_{ij} = A_{ji}$ (distance is symmetric), so the graph is **undirected**. Using an RBF weight is standard for building similarity graphs before spectral clustering.

(d) Degree matrix function

For an undirected weighted graph,

$$D = \operatorname{diag}\Big(\sum_j A_{1j}, \ldots, \sum_j A_{nj}\Big).$$

(This D is then later used to form $D^{-1/2}$ for normalization.)

(e) With $\gamma = 7.5$: build A, D, M

The matrix M was correctly computed as

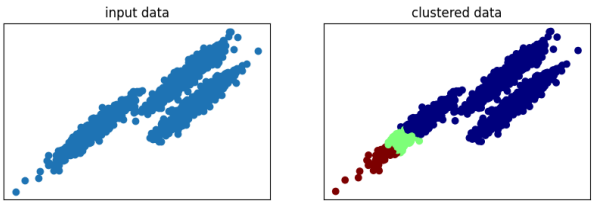
$$M = D^{-\frac{1}{2}} A D^{-\frac{1}{2}},$$

i.e., the **symmetrically normalized adjacency** (equivalently, $I - L_{\text{sym}}$ where $L_{\text{sym}} = I - D^{-1/2}AD^{-1/2}$ is the normalized Laplacian used in spectral methods).

(f) SVD on $M \rightarrow$ top-3 U vectors $\rightarrow$ KMeans

The function takes the top 3 left singular vectors ($U_{:,1:3}$) (row-normalized) and runs KMeans in that embedded space. In the plot produced for this step, the three diagonal bands are now cleanly separated—**it works**.

Why it works: Spectral clustering embeds points so that nodes strongly connected on the graph (high A_{ij}) become close in the low-dimensional eigenvector space; elongated manifolds that confound KMeans in $\mathbb{R}^2$ become almost spherical/compact in the spectral space, where KMeans performs well. This is the classic Ng–Jordan–Weiss recipe.



(g) Make M stochastic and repeat

M was normalized to a **stochastic** matrix by making columns sum to 1 (left/column-stochastic). In Markov-chain terms, "stochastic" just means each row sums to 1 (right-stochastic) or each column sums to 1 (left-stochastic), depending on convention. This column-normalization is valid, but different from the common random-walk (row-stochastic) normalization.

Running the same SVD→top-3→KMeans on the M_{stoch} plot, most points collapsed into one dominant cluster with tiny fragments elsewhere—**it worked poorly**. That's a symptom that this particular stochastic scaling distorted relative affinities (especially with dense RBF graphs). In practice, spectral clustering typically uses the (symmetric) $D^{-1/2}AD^{-1/2}$ embedding (as was done in (e)) or the **random-walk** normalization ($D^{-1}A$) and then eigenvectors of the corresponding (normalized) Laplacian, not an arbitrary column-stochastic rescaling.

---

6. Homework Process and Study Group

(a)

Based on the documents you provided, here are the 15 most important citations, covering the core topics of RNNs, algorithmic fairness, autoencoder theory, and practical Python environment issues.

Top 15 Most Important Citations:

1. Lecture Notes
2. **Learning Long-Term Dependencies with Gradient Descent is Difficult** (Bengio, Simard, Frasconi, 1994) - The foundational paper that first mathematically analyzed the vanishing and exploding gradients problem in recurrent neural networks.
3. **On the difficulty of training recurrent neural networks** (Pascanu, Mikolov, Bengio, 2013) - A key follow-up that provided a geometric interpretation of the problem and popularized the use of **gradient clipping** as a practical solution.
4. **Are Emily and Greg More Employable than Lakisha and Jamal?** (Bertrand & Mullainathan, 2004) - A landmark field experiment on racial discrimination in the labor market that has become a foundational text in the study of algorithmic bias.
5. **Neural Networks and Principal Component Analysis** (Baldi & Hornik, 1989) - The classic paper that formally proved the relationship between linear autoencoders and Principal Component Analysis (PCA).
6. **Deep Learning** (Goodfellow, Bengio, Courville, 2016) - One of the most comprehensive and widely-cited textbooks on modern deep learning, covering RNNs, CNNs, and more.
7. **Special category data** (UK Information Commissioner's Office) - The official guidance on what constitutes legally protected "special category data" under UK GDPR, essential for building fair and compliant ML systems.
8. **Matplotlib interactive widgets framework is not working in colab** (Google Colab GitHub Issue) - The official bug report acknowledging the core issue preventing interactive matplotlib backends like `ipympl` from working correctly in Google Colab.
9. **Auto-association by multilayer perceptrons and singular value decomposition** (Bourlard & Kamp, 1988) - Another foundational paper establishing the link between linear autoencoders and PCA via Singular Value Decomposition (SVD).
10. **EEOC says Workday must face claims that AI software is biased** (Reuters, 2024) - A significant, real-world legal case highlighting the regulatory and legal risks of deploying AI systems that may perpetuate discrimination.
11. **ipympl** (Matplotlib Documentation) - The official homepage and documentation for the `ipympl` backend, which provides interactive figures in Jupyter environments.
12. **Interactive matplotlib figures in Google Colab** (Stack Overflow) - A highly-referenced community discussion providing practical workarounds and context for the challenges of interactive plotting in Colab.
13. **Loss Landscapes of Regularized Linear Autoencoders** (Kunin et al., 2019) - A modern analysis that extends the classic understanding of linear autoencoders by examining the effects of regularization on their loss landscapes.
14. **Backends — Matplotlib Documentation** - The official explanation of Matplotlib's backend system, which is crucial for understanding why plotting behavior differs across environments like Colab and local machines.
15. **Proxy Attribute Discovery in Machine Learning Datasets** (Adler et al., 2021) - A research paper that addresses the critical fairness problem of identifying non-sensitive features (proxies) that are highly correlated with sensitive ones.
16. **Google Colab issue · Issue #587 · matplotlib/ipympl** (ipympl GitHub Issue) - A specific, detailed bug report from the `ipympl` repository discussing the incompatibility with Google Colab's environment.

(b)

None

(c)

1.5 day